Module 2: Data Analysis in Python & Data Visualization

Comprehensive Guide for Machine Learning and AI Freshers

Introduction

Data Analysis is the cornerstone of machine learning and artificial intelligence. Before building any predictive model or deploying an AI solution, you must understand, explore, and clean your data. This module equips you with essential tools and techniques to transform raw data into actionable insights.

The journey from raw data to insights involves three critical phases: **data manipulation** (NumPy and Pandas), **data visualization** (Matplotlib and Seaborn), and **exploratory data analysis** (EDA). These skills are fundamental for any AI/ML professional and form the backbone of successful data science projects in industry.

In this module, you'll master the libraries that data scientists use daily: **NumPy** for numerical computing, **Pandas** for data manipulation, **Matplotlib** and **Seaborn** for visualization, and **regular expressions** for text processing. By the end, you'll be able to extract insights from complex datasets—a skill that directly translates to solving real-world business problems.

Part 1: NumPy - Numerical Python

What is NumPy?

NumPy (Numerical Python) is the fundamental package for numerical computing in Python. It provides support for multi-dimensional arrays and matrices, along with a collection of mathematical functions. NumPy is the foundation upon which most scientific Python libraries (including Pandas, Scikit-learn, and TensorFlow) are built.

Why NumPy?

- **Performance**: NumPy operations are vectorized, making them 10-100x faster than pure Python
- **Memory efficiency**: NumPy arrays use less memory than Python lists
- **Convenience**: Broadcasting and vectorization make code more readable and Pythonic
- **Interoperability**: Works seamlessly with other scientific libraries

Introduction to Arrays

A **NumPy array** is a multi-dimensional collection of elements of the same data type, stored in contiguous memory. Unlike Python lists, NumPy arrays are optimized for numerical operations.

```
import numpy as np

# 1D array
array_1d = np.array([1, 2, 3, 4, 5])

# 2D array (matrix)
array_2d = np.array([[1, 2, 3],
                     [4, 5, 6],
                     [7, 8, 9]])

# 3D array
array_3d = np.array([[[1, 2], [3, 4]],
                     [[5, 6], [7, 8]]])

print(array_1d.shape)   # (5,)
print(array_2d.shape)   # (3, 3)
print(array_3d.shape)   # (2, 2, 2)
```

Creation and Printing of Arrays

```
import numpy as np

# Using np.array()
arr = np.array([1, 2, 3, 4, 5])

# Creating arrays with specific values
zeros = np.zeros((3, 3))      # All zeros
ones = np.ones((2, 4))         # All ones
empty = np.empty((3, 3))       # Uninitialized values

# Creating arrays with ranges
range_arr = np.arange(0, 10, 2)  # [0, 2, 4, 6, 8]
linspace_arr = np.linspace(0, 1, 5)  # 5 equally spaced values

# Creating random arrays
random_arr = np.random.rand(3, 3)     # Random [0, 1)
random_int = np.random.randint(0, 10, (3, 3))  # Random integers

# Creating identity matrix
identity = np.eye(3)

# Creating diagonal matrix
diagonal = np.diag([1, 2, 3])

# Print with formatting
print(array_2d)
print(f"Shape: {array_2d.shape}, Dtype: {array_2d.dtype}")
```

Basic Operations in NumPy

```
import numpy as np

arr1 = np.array([1, 2, 3, 4])
arr2 = np.array([5, 6, 7, 8])

# Element-wise operations
print(arr1 + arr2)        # [6, 8, 10, 12]
print(arr1 * arr2)        # [5, 12, 21, 32]
print(arr1 / arr2)        # Element-wise division
print(arr1 ** 2)          # Element-wise power

# Matrix operations
matrix_a = np.array([[1, 2], [3, 4]])
matrix_b = np.array([[5, 6], [7, 8]])

# Matrix multiplication
result = np.dot(matrix_a, matrix_b)
# or use @ operator
result = matrix_a @ matrix_b

# Transpose
transposed = matrix_a.T

# Trace (sum of diagonal elements)
trace = np.trace(matrix_a)

# Determinant
det = np.linalg.det(matrix_a)

# Inverse
inverse = np.linalg.inv(matrix_a)

# Aggregate operations
print(np.sum(arr1))       # Sum all elements
print(np.mean(arr1))      # Average
print(np.std(arr1))       # Standard deviation
print(np.max(arr1))       # Maximum
print(np.min(arr1))       # Minimum
```

Indexing in NumPy

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50])

# Basic indexing
print(arr[^0])             # 10 (first element)
print(arr[-1])            # 50 (last element)

# 2D array indexing
matrix = np.array([[1, 2, 3],
                   [4, 5, 6],
```

```
                    [7, 8, 9]])

print(matrix[0, 0])       # 1 (row 0, column 0)
print(matrix[1, 2])       # 6 (row 1, column 2)
print(matrix[2, :])       # [7, 8, 9] (entire row 2)
print(matrix[:, 1])       # [2, 5, 8] (entire column 1)

# Slicing
print(arr[1:4])           # [20, 30, 40]
print(arr[::2])           # [10, 30, 50] (every 2nd element)
print(matrix[0:2, 1:3])   # First 2 rows, columns 1-2

# Boolean indexing
mask = arr &gt; 25
print(arr[mask])          # [30, 40, 50]

# Fancy indexing
indices = np.array([0, 2, 4])
print(arr[indices])       # [10, 30, 50]
```

Mathematical Functions in NumPy

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])

# Trigonometric functions
print(np.sin(arr))        # Sine values
print(np.cos(arr))        # Cosine values
print(np.tan(arr))        # Tangent values

# Exponential and logarithmic
print(np.exp(arr))        # e^x for each element
print(np.log(arr))        # Natural logarithm
print(np.log10(arr))      # Base-10 logarithm
print(np.sqrt(arr))       # Square root

# Rounding
print(np.floor(np.array([1.5, 2.8, 3.2])))   # [1, 2, 3]
print(np.ceil(np.array([1.5, 2.8, 3.2])))    # [2, 3, 4]
print(np.round(np.array([1.5, 2.8, 3.2])))   # [2, 3, 3]

# Statistical functions
print(np.sum(arr))        # Sum
print(np.prod(arr))       # Product
print(np.cumsum(arr))     # Cumulative sum: [1, 3, 6, 10, 15]
print(np.cumprod(arr))    # Cumulative product

# Sorting
unsorted = np.array([3, 1, 4, 1, 5, 9])
print(np.sort(unsorted))  # [1, 1, 3, 4, 5, 9]
print(np.argsort(unsorted))  # Indices that would sort the array
```

Part 2: Pandas - Data Manipulation

Introduction to Series and DataFrames

Series: A 1D labeled array that can hold any data type.
DataFrame: A 2D labeled table-like structure with rows and columns (similar to SQL tables or Excel spreadsheets).

```
import pandas as pd

# Creating a Series
series = pd.Series([10, 20, 30, 40], index=['a', 'b', 'c', 'd'])
print(series)
print(series['b'])        # 20

# Creating a DataFrame
data = {'Name': ['Alice', 'Bob', 'Charlie'],
        'Age': [25, 30, 28],
        'Salary': [50000, 60000, 55000]}
df = pd.DataFrame(data)

# Check first/last rows
print(df.head())
print(df.tail())

# Basic info
print(df.info())          # Data types and missing values
print(df.shape)           # (3, 3)
print(df.describe())      # Statistical summary
```

Data Importing and Exporting

```
import pandas as pd

# Reading CSV
df = pd.read_csv('data.csv')

# Reading Excel
df = pd.read_excel('data.xlsx', sheet_name='Sheet1')

# Reading from different sources
df = pd.read_json('data.json')
df = pd.read_sql('SELECT * FROM table', connection)
df = pd.read_html('https://example.com/table.html')[^0]

# Writing to CSV
df.to_csv('output.csv', index=False)

# Writing to Excel
df.to_excel('output.xlsx', sheet_name='Sheet1')

# Writing to JSON
df.to_json('output.json', orient='records')
```

```
# Writing to SQL
df.to_sql('table_name', connection, if_exists='append')
```

Data Understanding Operations

```
import pandas as pd

df = pd.read_csv('data.csv')

# Shape and size
print(f"Rows: {df.shape[^0]}, Columns: {df.shape[^1]}")

# Column names and types
print(df.columns)
print(df.dtypes)

# Statistical summary
print(df.describe())      # Numeric columns only
print(df.describe(include='all'))  # All columns

# Value counts
print(df['Category'].value_counts())

# Unique values
print(df['Category'].nunique())
print(df['Category'].unique())

# Missing values
print(df.isnull().sum())  # Count of missing values per column
print(df.isnull().sum().sum())  # Total missing values

# Correlation matrix
print(df.corr())
```

Indexing, Slicing, and Filtering

```
import pandas as pd

df = pd.DataFrame({'Name': ['Alice', 'Bob', 'Charlie'],
                   'Age': [25, 30, 28],
                   'City': ['NYC', 'LA', 'NYC']})

# Label-based indexing
print(df.loc[^0])          # First row
print(df.loc[0, 'Name'])  # Specific cell

# Integer-based indexing
print(df.iloc[^0])         # First row
print(df.iloc[0:2])       # First two rows

# Filtering with Boolean conditions
print(df[df['Age'] &gt; 26])
```

```
# Multiple conditions
print(df[(df['Age'] &gt; 25) &amp; (df['City'] == 'NYC')])

# Using query() for complex conditions
print(df.query('Age &gt; 25 and City == "NYC"'))

# Filtering with isin()
print(df[df['City'].isin(['NYC', 'LA'])])
```

GroupBy, Pivot Table, and Cross Tab

```
import pandas as pd

df = pd.DataFrame({'Department': ['Sales', 'Sales', 'IT', 'IT', 'HR'],
                   'Employee': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve'],
                   'Salary': [50000, 55000, 60000, 65000, 48000]})

# GroupBy - aggregate by group
print(df.groupby('Department')['Salary'].mean())
print(df.groupby('Department').agg({'Salary': ['mean', 'sum', 'count']}))

# Pivot Table
df_pivot = pd.pivot_table(df, values='Salary', index='Department', aggfunc='mean')
print(df_pivot)

# Cross Tab
df_data = pd.DataFrame({'Gender': ['M', 'F', 'M', 'F'],
                        'Age_Group': ['20-30', '20-30', '30-40', '30-40']})
print(pd.crosstab(df_data['Gender'], df_data['Age_Group']))
```

Concatenating, Merging, and Joining

```
import pandas as pd

df1 = pd.DataFrame({'A': [1, 2], 'B': [3, 4]})
df2 = pd.DataFrame({'A': [5, 6], 'B': [7, 8]})

# Concatenation (stacking)
df_concat = pd.concat([df1, df2], ignore_index=True)

# Merge (like SQL JOIN)
df_left = pd.DataFrame({'Key': ['A', 'B'], 'Value1': [1, 2]})
df_right = pd.DataFrame({'Key': ['A', 'B'], 'Value2': [3, 4]})

merged = pd.merge(df_left, df_right, on='Key', how='inner')
# how can be 'inner', 'outer', 'left', 'right'

# Join (merge on index)
df_join = df1.join(df2, rsuffix='_right')
```

Descriptive Statistics

```
import pandas as pd

df = pd.read_csv('data.csv')

# Central tendency
print(df.mean())          # Average
print(df.median())        # Median
print(df.mode())          # Mode

# Dispersion
print(df.std())           # Standard deviation
print(df.var())           # Variance
print(df.min())           # Minimum
print(df.max())           # Maximum
print(df.quantile(0.25))  # 25th percentile

# Skewness and Kurtosis
print(df.skew())          # Skewness
print(df.kurtosis())      # Kurtosis

# Correlation
print(df.corr())          # Pearson correlation
print(df.corr(method='spearman'))  # Spearman correlation
```

Removing Duplicates

```
import pandas as pd

df = pd.DataFrame({'Name': ['Alice', 'Bob', 'Alice', 'Charlie'],
                   'Age': [25, 30, 25, 28]})

# Identify duplicates
print(df.duplicated())

# Remove duplicates
df_clean = df.drop_duplicates()

# Remove duplicates based on specific columns
df_clean = df.drop_duplicates(subset=['Name'])

# Keep first or last occurrence
df_clean = df.drop_duplicates(keep='first')  # or 'last'
```

String Manipulation

```
import pandas as pd

df = pd.DataFrame({'Name': ['alice smith', 'bob JONES', 'charlie brown']})

# String methods
```

```
print(df['Name'].str.upper()      # Convert to uppercase
print(df['Name'].str.lower()      # Convert to lowercase
print(df['Name'].str.len()        # Length of each string
print(df['Name'].str.split()      # Split strings
print(df['Name'].str.replace('a', '@'))  # Replace characters

# Extract and contains
print(df['Name'].str.contains('alice'))
print(df['Name'].str.extract(r'(\w+)\s(\w+)'))  # Extract first and last names
```

Missing Data Handling

```
import pandas as pd

df = pd.DataFrame({'A': [1, 2, None, 4],
                   'B': [5, None, None, 8]})

# Detect missing values
print(df.isnull())
print(df.isnull().sum())

# Remove rows/columns with missing values
df_clean = df.dropna()              # Remove rows with any NaN
df_clean = df.dropna(how='all')     # Remove rows where all are NaN
df_clean = df.dropna(thresh=2)      # Keep rows with at least 2 non-NaN values

# Fill missing values
df_filled = df.fillna(0)            # Fill with constant
df_filled = df.fillna(df.mean())    # Fill with mean
df_filled = df.fillna(method='ffill')  # Forward fill
df_filled = df.fillna(method='bfill')  # Backward fill

# Interpolation
df_filled = df.interpolate()
```

Part 3: Data Visualization

Introduction to Matplotlib

Matplotlib is the most flexible and widely-used visualization library in Python. It provides a MATLAB-like interface and is the foundation for other libraries like Seaborn.

```
import matplotlib.pyplot as plt
import numpy as np

# Basic plot
x = np.array([1, 2, 3, 4, 5])
y = np.array([2, 4, 6, 8, 10])

plt.plot(x, y)
plt.title("Simple Line Plot")
```

```
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.show()
```

Basic Plotting

```
import matplotlib.pyplot as plt
import numpy as np

# Line plot
plt.figure(figsize=(10, 6))
x = np.linspace(0, 10, 100)
y = np.sin(x)
plt.plot(x, y, label='sin(x)', linewidth=2)
plt.plot(x, np.cos(x), label='cos(x)', linewidth=2)
plt.legend()
plt.title("Trigonometric Functions")
plt.xlabel("x")
plt.ylabel("y")
plt.grid(True)
plt.show()

# Scatter plot
plt.scatter([1, 2, 3, 4], [2, 4, 5, 3], s=100, alpha=0.5)
plt.title("Scatter Plot")
plt.show()
```

Properties of Plotting

```
import matplotlib.pyplot as plt

# Line styles, markers, colors
plt.plot([1, 2, 3], [1, 2, 3], linestyle='-', linewidth=2, color='red')
plt.plot([1, 2, 3], [3, 2, 1], linestyle='--', marker='o', color='blue')
plt.plot([1, 2, 3], [2, 2, 2], linestyle=':', marker='^', color='green')

# Color options: 'r', 'g', 'b', 'y', 'k', 'w', or hex codes
# Linestyle: '-', '--', '-.', ':'
# Marker: 'o', 's', '^', 'D', '*', '+', 'x'

plt.legend(['Line', 'Dashed', 'Dotted'])
plt.show()
```

Subplots

```
import matplotlib.pyplot as plt
import numpy as np

# Create 2x2 subplots
fig, axes = plt.subplots(2, 2, figsize=(10, 8))
```

```
# Plot in each subplot
axes[0, 0].plot([1, 2, 3], [1, 2, 3])
axes[0, 0].set_title("Plot 1")

axes[0, 1].scatter([1, 2, 3], [1, 2, 3])
axes[0, 1].set_title("Plot 2")

axes[1, 0].bar([1, 2, 3], [1, 2, 3])
axes[1, 0].set_title("Plot 3")

axes[1, 1].hist([1, 1, 2, 2, 2, 3, 3, 3, 3], bins=3)
axes[1, 1].set_title("Plot 4")

plt.tight_layout()
plt.show()
```

Line Plots, Pie Charts, and Bar Graphs

```
import matplotlib.pyplot as plt

# Line plot
months = ['Jan', 'Feb', 'Mar', 'Apr', 'May']
sales = [100, 150, 120, 200, 180]
plt.plot(months, sales, marker='o')
plt.title("Monthly Sales")
plt.show()

# Pie chart
categories = ['A', 'B', 'C', 'D']
values = [30, 25, 20, 25]
plt.pie(values, labels=categories, autopct='%1.1f%%', startangle=90)
plt.title("Distribution")
plt.show()

# Bar graph
plt.bar(categories, values, color=['red', 'blue', 'green', 'yellow'])
plt.title("Category Values")
plt.ylabel("Values")
plt.show()
```

Histograms

```
import matplotlib.pyplot as plt
import numpy as np

data = np.random.normal(100, 15, 1000)

# Histogram
plt.hist(data, bins=30, edgecolor='black', alpha=0.7)
plt.title("Distribution of Data")
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.axvline(np.mean(data), color='red', linestyle='--', label='Mean')
```

```
plt.axvline(np.median(data), color='green', linestyle='--', label='Median')
plt.legend()
plt.show()
```

Box and Violin Plots

```
import matplotlib.pyplot as plt
import numpy as np

# Generate sample data
data = [np.random.normal(0, std, 100) for std in range(1, 4)]

# Box plot
plt.boxplot(data, labels=['A', 'B', 'C'])
plt.title("Box Plot")
plt.ylabel("Values")
plt.show()

# Violin plot
parts = plt.violinplot(data, positions=[1, 2, 3])
plt.title("Violin Plot")
plt.xticks([1, 2, 3], ['A', 'B', 'C'])
plt.show()
```

Scatterplots

```
import matplotlib.pyplot as plt
import numpy as np

# Generate random data
x = np.random.rand(100)
y = np.random.rand(100)
colors = np.random.rand(100)
sizes = np.random.rand(100) * 100

# Scatter plot
plt.scatter(x, y, c=colors, s=sizes, alpha=0.5, cmap='viridis')
plt.colorbar(label='Color Value')
plt.title("Scatter Plot with Colors and Sizes")
plt.xlabel("X")
plt.ylabel("Y")
plt.show()
```

Part 4: Seaborn - Statistical Visualization

Introduction to Seaborn

Seaborn is built on top of Matplotlib and provides a high-level interface for drawing attractive and informative statistical graphics. It works seamlessly with Pandas DataFrames.

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

# Load sample dataset
df = sns.load_dataset('tips')

# Basic scatter plot with Seaborn
sns.scatterplot(data=df, x='total_bill', y='tip', hue='sex', s=100)
plt.show()
```

Seaborn Plots

```
import seaborn as sns
import matplotlib.pyplot as plt

# Load data
df = sns.load_dataset('tips')

# Line plot
sns.lineplot(data=df, x='size', y='total_bill')
plt.show()

# Count plot (bar plot for categorical data)
sns.countplot(data=df, x='day', hue='sex')
plt.show()

# Heatmap
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm')
plt.show()

# Pair plot (scatter plot matrix)
sns.pairplot(df, hue='sex')
plt.show()

# Categorical plot
sns.catplot(data=df, x='day', y='total_bill', kind='box', hue='sex')
plt.show()
```

Part 5: Exploratory Data Analysis (EDA)

What is EDA?

Exploratory Data Analysis is the process of examining datasets to summarize their main characteristics, often using visual methods. EDA helps identify patterns, detect anomalies, and understand relationships before formal modeling.

EDA Workflow:

1. **Data Loading** - Import and inspect the dataset
2. **Data Cleaning** - Handle missing values, duplicates, and outliers
3. **Univariate Analysis** - Analyze individual variables
4. **Bivariate Analysis** - Examine relationships between pairs of variables
5. **Multivariate Analysis** - Explore relationships among multiple variables
6. **Insights and Recommendations** - Summarize findings

Univariate Analysis

Univariate analysis examines the distribution of a single variable.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv('data.csv')

# Numeric variable analysis
print(df['Age'].describe())

# Visualizations for numeric data
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

# Histogram
axes[^0].hist(df['Age'], bins=20, edgecolor='black')
axes[^0].set_title("Histogram of Age")

# Box plot
axes[^1].boxplot(df['Age'])
axes[^1].set_title("Box Plot of Age")

# KDE plot
sns.kdeplot(data=df, x='Age', ax=axes[^2])
axes[^2].set_title("KDE Plot of Age")

plt.tight_layout()
plt.show()

# Categorical variable analysis
print(df['Category'].value_counts())

# Bar plot for categorical
sns.countplot(data=df, x='Category')
```

```
plt.title("Count of Categories")
plt.show()
```

Bivariate Analysis

Bivariate analysis examines relationships between two variables.

```
import seaborn as sns
import matplotlib.pyplot as plt

df = sns.load_dataset('tips')

# Numeric vs Numeric
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
sns.scatterplot(data=df, x='total_bill', y='tip')
plt.title("Total Bill vs Tip")

plt.subplot(1, 2, 2)
sns.regplot(data=df, x='total_bill', y='tip')
plt.title("Total Bill vs Tip (with regression line)")
plt.tight_layout()
plt.show()

# Categorical vs Numeric
sns.boxplot(data=df, x='day', y='total_bill', hue='sex')
plt.title("Total Bill by Day and Sex")
plt.show()

# Categorical vs Categorical
pd.crosstab(df['day'], df['sex'])
sns.heatmap(pd.crosstab(df['day'], df['sex']), annot=True, fmt='d')
plt.show()
```

Advanced Seaborn Plots for EDA

```
import seaborn as sns
import matplotlib.pyplot as plt

df = sns.load_dataset('iris')

# Pair plot - scatter plot matrix
sns.pairplot(df, hue='species')
plt.show()

# Correlation heatmap
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm', center=0)
plt.title("Correlation Matrix")
plt.show()

# Categorical plot with multiple representations
sns.catplot(data=df, x='species', y='petal_length', kind='violin', hue='species')
plt.show()
```

```
# Joint plot - bivariate distribution
sns.jointplot(data=df, x='sepal_length', y='petal_length', kind='hex')
plt.show()

# Count plot
sns.countplot(data=df, x='species', palette='Set2')
plt.title("Distribution of Species")
plt.show()
```

Part 6: Regular Expressions and Web Scraping

Introduction to Regular Expressions

Regular Expressions (Regex) are patterns used to match and manipulate strings. They're powerful for text mining, data cleaning, and validation.

```
import re

# Basic pattern matching
text = "The email is alice@example.com"
pattern = r'\w+@\w+\.\w+'

match = re.search(pattern, text)
if match:
    print(f"Found: {match.group()}")

# Find all matches
emails = re.findall(r'\w+@\w+\.\w+', "Contact: alice@ex.com or bob@test.org")
print(emails)  # ['alice@ex.com', 'bob@test.org']
```

Common Regex Patterns

```
import re

# Metacharacters and literals
text = "abc123def456"

# . matches any character (except newline)
print(re.findall(r'a.c', text))

# \d matches digits, \w matches word characters, \s matches whitespace
print(re.findall(r'\d+', text))  # ['123', '456']

# Quantifiers
print(re.findall(r'\d{3}', text))     # Exactly 3 digits
print(re.findall(r'[0-9]+', text))    # One or more digits

# Character classes
print(re.findall(r'[a-z]+', text))    # Lowercase letters
print(re.findall(r'[A-Za-z]+', text))  # All letters
```

```
# Anchors
print(re.findall(r'^a', text))        # Starts with 'a'
print(re.findall(r't$', 'test'))      # Ends with 't'
```

Using Regex in Pandas

```
import pandas as pd
import re

df = pd.DataFrame({'Email': ['alice@ex.com', 'invalid-email', 'bob@test.org']})

# Check if pattern matches
df['is_valid_email'] = df['Email'].str.match(r'\w+@\w+\.\w+')

# Extract part of string
df['domain'] = df['Email'].str.extract(r'@(\w+\.\w+)')

# Replace using regex
df['Email_masked'] = df['Email'].str.replace(r'(\w).+(@)', r'\1***\2')

print(df)
```

Web Scraping with BeautifulSoup

```
import requests
from bs4 import BeautifulSoup
import pandas as pd

# Step 1: Fetch webpage
url = 'https://example.com/products'
response = requests.get(url)

# Step 2: Parse HTML
soup = BeautifulSoup(response.content, 'html.parser')

# Step 3: Extract data
products = []
for item in soup.find_all('div', class_='product'):
    name = item.find('h2').text
    price = item.find('span', class_='price').text
    products.append({'Name': name, 'Price': price})

# Step 4: Convert to DataFrame
df = pd.DataFrame(products)
print(df)
```

Complete Web Scraping Project

```
import requests
from bs4 import BeautifulSoup
import pandas as pd
import re

def scrape_data(url):
    """Scrape data from website"""
    response = requests.get(url)
    soup = BeautifulSoup(response.content, 'html.parser')
    return soup

def extract_information(soup):
    """Extract structured data from HTML"""
    items = []
    for row in soup.find_all('tr')[1:]:  # Skip header
        cells = row.find_all('td')
        item = {
            'Name': cells[^0].text.strip(),
            'Price': float(re.search(r'\d+\.\d+', cells[^1].text).group()),
            'Date': cells[^2].text.strip()
        }
        items.append(item)
    return items

def clean_data(items):
    """Clean and transform data"""
    df = pd.DataFrame(items)
    df['Price'] = df['Price'].astype(float)
    df = df.drop_duplicates()
    return df

# Execute
url = 'https://example.com/data'
soup = scrape_data(url)
data = extract_information(soup)
df = clean_data(data)
print(df)
```

Case Study: Complete EDA Analysis

A practical example of a complete EDA process:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# 1. Load data
df = pd.read_csv('ecommerce_data.csv')

# 2. Initial inspection
```

```
print(f"Dataset shape: {df.shape}")
print(f"\nMissing values:\n{df.isnull().sum()}")
print(f"\nData types:\n{df.dtypes}")

# 3. Data cleaning
df = df.dropna(subset=['Price', 'Rating'])  # Remove critical missing values
df = df.drop_duplicates()
df['Price'] = pd.to_numeric(df['Price'], errors='coerce')

# 4. Univariate analysis
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[^0].hist(df['Price'], bins=30, edgecolor='black')
axes[^0].set_title("Distribution of Price")
axes[^1].hist(df['Rating'], bins=10, edgecolor='black')
axes[^1].set_title("Distribution of Rating")
plt.tight_layout()
plt.show()

# 5. Bivariate analysis
sns.scatterplot(data=df, x='Price', y='Rating')
plt.title("Price vs Rating")
plt.show()

# 6. Multivariate analysis
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm')
plt.title("Correlation Matrix")
plt.show()

# 7. Summary statistics
print("\nSummary Statistics:")
print(df.describe())

# 8. Key insights
print(f"\nAverage Price: ${df['Price'].mean():.2f}")
print(f"Average Rating: {df['Rating'].mean():.2f}")
print(f"Most Common Category: {df['Category'].mode()[^0]}")
```

References and Further Learning

1. Harris, C. R., et al. (2024). NumPy: A Foundational Library for Scientific Computing. Nature Reviews Methods Primers, 2(1), 1-18.
2. McKinney, W. (2022). Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython (3rd ed.). O'Reilly Media.
3. Waskom, M. L. (2024). Seaborn: Statistical Data Visualization. Journal of Open Source Software, 6(60), 3021.
4. Plotly Technologies Inc. (2024). Plotly Graphing Libraries. Retrieved from https://plotly.com/
5. Requests Library Documentation. (2024). Kenneth Reitz & Phil Sturgeon. Retrieved from https://requests.readthedocs.io/
6. BeautifulSoup Documentation. (2024). Leonard Richardson. Retrieved from https://www.crummy.com/software/BeautifulSoup/

7. GeeksforGeeks. (2024). Complete Guide to NumPy and Pandas. Retrieved from https://www.geeksforgeeks.org/
8. Real Python. (2024). Web Scraping with Python. Retrieved from https://realpython.com/
9. Pandas Development Team. (2024). Pandas: Powerful Data Structures for Data Analysis. Retrieved from https://pandas.pydata.org/
10. Chollet, F., & Allaire, J. J. (2023). Deep Learning with Python (2nd ed.). Manning Publications.

Industry Best Practices

- **Always explore before modeling** - EDA prevents model failures due to data issues
- **Vectorize operations** - Use NumPy and Pandas vectorization instead of loops
- **Document data quality** - Track missing values, outliers, and data transformations
- **Automate EDA** - Use pandas-profiling for quick dataset summaries
- **Version control data pipelines** - Track changes to data processing scripts
- **Validate scraped data** - Always verify web scraping results for accuracy
- **Respect robots.txt** - Follow website scraping guidelines
- **Handle missing data thoughtfully** - Choose methods (deletion/imputation) based on context
[1] [2] [3] [4] [5] [6] [7] [8] [9] [10] [11] [12] [13] [14] [15] [16] [17] [18] [19] [20] [21] [22] [23] [24] [25] [26] [27] [28]

⁂

1. https://pyseek.com/2024/09/seaborn-in-python/
2. https://www.investopedia.com/terms/d/descriptive_statistics.asp
3. https://www.almabetter.com/bytes/tutorials/gate-data-science-and-ai/descriptive-statistics-for-gate
4. https://economia.uniroma2.it/ba/globalgovernance/corso/asset/YTo0OntzOjI6ImlkIjtzOjQ6IjI1ODYiO3M6MzoiaWRhIjtzOjU6Ijc0MDI5IjtzOjI6ImVtIjtOO3M6MToiYyI7czo1OiIxNmI1NyI7fQ==
5. https://ledidi.com/academy/measures-of-central-tendency-mean-median-and-mode
6. https://www.dissertationindia.com/blog/post/what-is-descriptive-statistics-and-how-to-use-it
7. https://www.geeksforgeeks.org/maths/binomial-distribution/
8. https://www.scribd.com/document/589259400/Test-of-Hypothesis-t-and-Z-tests-Chi-square-Test-F-Test
9. https://www.simplilearn.com/what-is-descriptive-statistics-article
10. https://www.healthknowledge.org.uk/public-health-textbook/research-methods/1b-statistical-methods/statistical-distributions
11. https://www.datacamp.com/tutorial/t-test-vs-z-test
12. https://serpapi.com/blog/beautiful-soup-build-a-web-scraper-with-python/
13. https://realpython.com/beautiful-soup-web-scraper-python/
14. https://www.browserstack.com/guide/web-scraping-using-beautiful-soup
15. https://www.scrapingbee.com/blog/web-scraping-101-with-python/
16. https://www.scraperapi.com/web-scraping/beautiful-soup/
17. https://emeritus.org/in/learn/exploratory-data-analysis/

18. https://developers.google.com/edu/python/regular-expressions
19. https://www.promptcloud.com/blog/web-scraping-with-python/
20. https://www.linkedin.com/pulse/exploratory-data-analysis-eda-case-study-archana-khewariya-jqquf
21. https://expediteinformatics.com/exploring-numpy-in-2024-trends-standards-benefits-challenges-and-commitments/
22. https://www.geeksforgeeks.org/python/pattern-matching-python-regex/
23. https://www.projectpro.io/article/numpy-cheatsheet/922
24. https://realpython.com/numpy-example/
25. https://www.dasca.org/world-of-data-science/article/unlocking-data-manipulation-in-python-key-pandas-techniques
26. https://www.analytixlabs.co.in/blog/python-visualization/
27. https://numpy.org/doc/2.2/user/absolute_beginners.html
28. https://codebasics.io/blog/mastering-data-manipulation-with-pandas-a-comprehensive-guide-for-aspiring-data-scientists